# ANTHOLOGY DASHBOARD APPLICATION

## Version 1.0


Created by : Sreenivasulu Puli & Chaitanya Deshmukh

Reviewed by : Satish Bindumadhavan

Approver : **Rawlings, Christopher**

## Revision History

| Ver. No. | Date | Sec. No. | Amendments Made | Change Request by | Reviewed and Approved By |
|---|---|---|---|---|---|
| 1.0 | 23rd Oct,2025 | All | Initial release | | Reviewed: Satish Bindumadhavan |

# Table of Contents

# 1. Definitions

Below are the key terms and concepts used in the document

| Term | Definition |
|---|---|
| **Anthology Dashboard Application** | A centralized web-based platform designed to manage, review, and approve Monitoring Shave Test (MST) QA reports for Gillette's Blades & Razors capability. |
| **Monitoring Shave Test (MST)** | A QA process conducted by Gillette to assess manufacturing quality based on consumer response panels comparing production samples to calibration stock. |
| **PingFederate SSO (SAML 2.0)** | A Single Sign-On solution that authenticates users using P&G's corporate credentials through Security Assertion Markup Language (SAML) protocol. |
| **QA Approver** | A Quality Assurance professional authorized to review draft reports, assign Pass/Fail status, and manage report recipients within the dashboard. |
| **Manufacturing SPOC** | A Single Point of Contact responsible for reviewing finalized QA reports for assigned manufacturing sites and ensuring corrective actions when required. |
| **Admin User** | A system administrator responsible for viewing logs, monitoring activities, and maintaining audit trails, without performing approvals. |
| **Azure Blob Storage** | Microsoft Azure's object storage service is used to securely store draft and final QA reports for PDFs accessed by the application. |
| **LDAP (Lightweight Directory Access Protocol)** | A directory service protocol used for managing and retrieving email distribution lists and user group information from P&G's internal network. |
| **MySQL Database** | A relational database used to store metadata related to reports, approvals, users, and system logs for traceability and reporting. |
| **Report Metadata** | Supplementary data describing each QA report, including report ID, site, region, status, approver, and timestamps. |
| **Role-Based Access Control (RBAC)** | A security mechanism that restricts system access based on predefined user roles (QA Approver, SPOC, Admin). |
| **Pass/Fail Status** | The outcome of QA report evaluation where "Pass" indicates compliance with standards, and "Fail" denotes deviation or quality concern. |

| | |
|---|---|
| **Audit Log** | A chronological record capturing user activities, approvals, and system events for compliance, traceability, and auditing. |
| **Dashboard UI** | The user interface of the Anthology Dashboard Application presents reports, metrics, and controls for authorized users. |
| **Gunicorn** | A Python WSGI HTTP server used to deploy Django applications in a production environment. |
| **Nginx** | A web server is used as a reverse proxy to handle client requests, serve static files, and improve application security and performance. |
| **Django Framework** | A high-level Python web framework used for rapid, secure, and maintainable application development. |
| **Single Source of Truth (SSOT)** | A data management concept where the Anthology Dashboard serves as the authoritative and centralized repository for all QA reports. |
| **VPN (Virtual Private Network)** | A secure connection method that allows developers and authorized users to access internal P&G systems remotely. |
| **UAT (User Acceptance Testing)** | The validation phase where client stakeholders review and approve the application before production deployment. |
| **Deployment Server** | The on-premises Ubuntu server environment where the Anthology Dashboard Application will be hosted and maintained. |
| **Sprint** | A defined development cycle, typically lasting one week, during which specific deliverables or features are designed, built, and tested. |

## 2. References

| SL. No. | Document Title |
|---|---|
| 1 | Anthology_Dashboard_App_SOW.docx |
| 2 | Data_Model.docx |
| 3 | Wireframe.drawio.pdf |

# 3. Executive Summary

## 3.1 Project Overview

The Anthology Dashboard Application is a web-based solution designed to streamline and centralize the Quality Assurance (QA) reporting process for the Monitoring Shave Test (MST) conducted by Gillette's *Blades & Razors* capability.

Currently, QA reports are assembled manually from multiple data sources, requiring significant manual intervention for data processing, formatting, and distribution. The Anthology Dashboard addresses these inefficiencies by automating the flow of data, enabling real-time access, and ensuring standardized storage of reports across all manufacturing sites.

This Django-based web application will:

- Integrate with PingFederate (SSO - SAML 2.0) for secure authentication.
- Fetch reports from Azure Blob Storage and manage recipient data through LDAP integration.
- Store report metadata and process logs in an on-premises MySQL database.
- Provide role-based access for QA Approvers, Manufacturing SPOCs, and Administrators.
- Offer a clean, responsive Bootstrap-based UI for browsing, approving, and monitoring reports.
- Maintain detailed audit logs for compliance and traceability.

By centralizing QA report access and automating distribution, the Anthology Dashboard enhances efficiency, transparency, and governance while minimizing manual workload and inconsistencies across regional QA operations.

## 3.2 Scope of the Project

The scope of the Anthology Dashboard Application includes the design, development, integration, and deployment of a secure, role-based web platform that centralizes access to QA reports generated through the Monitoring Shave Test (MST) process.

**In Scope Activities**

1. **Application Development**
    a. Build a Django-based web application hosted on an on-premises Ubuntu server.
    b. Implement role-based access control (RBAC) for three user types:
        i. QA Approver – Full access to reports, approval actions, and log monitoring.
            1. Display daily report count summary for QA users upon login.
        ii. Manufacturing SPOC – Restricted access to finalized reports for assigned sites / regions.
        iii. Admin – Access to logs and historical data, without 'Approval' privileges.
2. **Integration Components**
    a. **PingFederate (SSO – SAML 2.0):** For federated authentication and dynamic role assignment.
    b. **Azure Blob Storage:** For secure retrieval of draft and final QA report PDFs.
    c. **LDAP Integration:** To fetch and manage report recipient email distribution lists.
    d. **MySQL Database:** For storing report metadata and approval records.
3. **User Interface and Functionality**
    a. Develop an intuitive Bootstrap 5 – based UI for report browsing, filtering, and viewing.
    b. Enable report approval workflow using Pass/Fail logic (replacing Approve/Reject terminology).
    c. Include dashboard summaries showing key QA metrics, pending approvals, and site activity.
4. **System Operations**
    a. Ensure secure deployment and configuration on an on-premises Ubuntu environment.
    b. Track and log PDF downloads and report access for audit purposes.

## 3.3 Out of Scope

**The following features are considered out of scope for the current delivery:**

1. Report generation or modification of data pipelines (handled by existing MST systems).
2. Manual creation or editing of reports within the dashboard.
3. Mobile or offline access to the application.
4. Long-term data archival beyond the application's storage layer.
5. Email distribution handling (only recipient management supported).
6. Administration of user roles, sites, and regions within PingFederate or LDAP (consumed as provided).

## 3.4 Project Assumptions and Dependencies

### 3.4.1 Project Assumptions

The successful execution of the Anthology Dashboard Application development is based on the following key assumptions:

1. **Access and Infrastructure**
    a. The on-premises Ubuntu server environment will be provisioned, configured, and accessible to the development team prior to the start of implementation.
    b. Necessary VPN access and intranet credentials will be provided to the developers for connecting to internal resources (PingFederate, LDAP, MySQL, and Azure Blob Storage, VS code).
    c. Adequate storage, and network resources will be available on the hosting server to support application deployment and testing.
2. **Integration Readiness**
    a. The PingFederate SSO (SAML 2.0) environment will be configured and operational, with relevant metadata and attribute mappings (roles, sites, regions) shared with the development team.
    b. The Azure Blob Storage containers (for draft and final reports) will be available with read access via SAS token or service principal.
    c. The LDAP directory and MySQL database instances will be accessible for testing and integration purposes during development.
    d. **PingFederate SSO** will be the **only authentication method in production**.
        i. During development or early testing, a **fallback debug login** allows manual user simulation without real authentication.
        ii. **Login interface** is pre-configured to redirect to the SSO provider once details are available.
        iii. **Public certificate storage** and validation logic are built in, with assumptions documented.
        iv. **SAML assertion endpoints** are defined and ready for integration with Identity Services.
        v. **Configuration values** can be updated during deployment as needed.
        **vi.** This approach ensures **secure production access** while supporting **ongoing development** even if SSO onboarding is delayed.
3. **Data and Content**
    a. Sample PDF reports and data structures (metadata fields, report naming conventions, etc.) will be provided to enable realistic testing of the dashboard UI.
    b. User role mappings, site assignments, and region-level data will be accurately defined in the PingFederate/LDAP systems and kept up to date.
    c. Metadata should include fields for lock status and user activity to support UI indicators and access control
4. **Stakeholder Engagement**

a. The client stakeholders (Christopher and Avery) will be available for periodic reviews, approvals, and clarifications during the weekly sync meetings.
b. Timely feedback will be provided during sprint demos and UAT cycles to avoid delays.

5. **Development Environment**
   a. The development will occur directly on the designated Ubuntu server within the P&G network, ensuring environmental parity with production.
   b. All third-party dependencies and libraries required by Django and its integrations will be permitted for installation.
6. **Security and Compliance**
   a. All integrations and code implementations will adhere to P&G's internal IT security standards, including authentication, encryption, and data handling protocols.
   b. No external or third-party API calls will be made outside approved systems (PingFederate, Azure, LDAP, MySQL).

### 3.4.2 Project Dependencies

The completion and delivery of the project are dependent on the following external and internal factors:

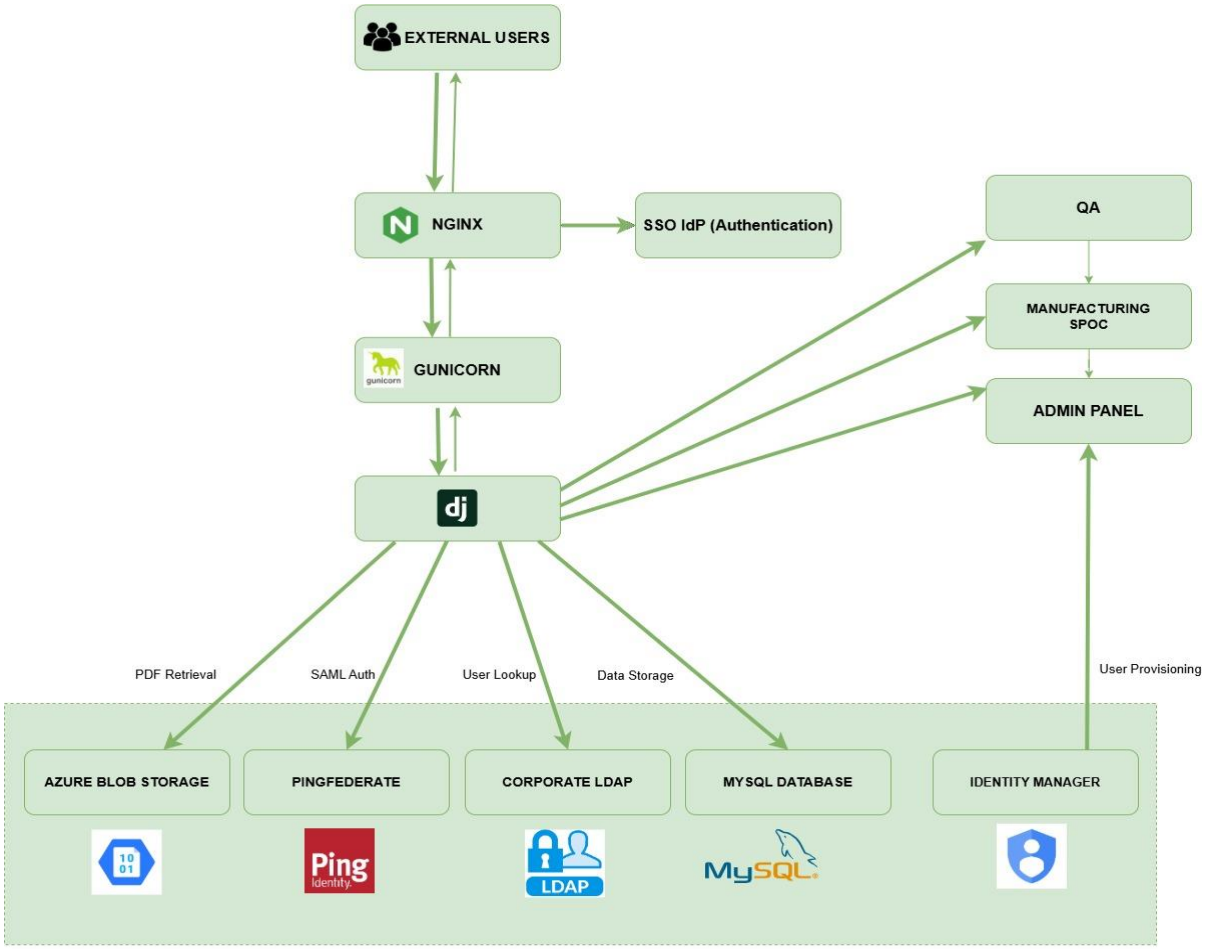
1. **System Availability**
   a. Continuous uptime and accessibility of PingFederate, LDAP, Azure Blob Storage, and MySQL services are essential for integration testing and deployment.
   b. Identity Manager must remain accessible for user provisioning and role assignment validation.
2. **Credential and Access Provisioning**
   a. Timely creation of developer accounts, VPN configurations, and GitHub repository access under the P&G domain are critical for project initiation.
3. **Client Approvals**
   a. Design reviews, sprint sign-off, and UAT approvals from the client team are required to progress to subsequent stages of development.
4. **External Data Pipeline**
   a. The data processing pipeline (outside the project scope) that generates report PDFs must remain stable and consistent in structure and delivery format.
5. **IT Infrastructure Support**
   a. Coordination with P&G's IT infrastructure and security teams will be required for SSL certificate installation, firewall permissions, and production deployment.
6. **Third-party Tool Dependencies**
   a. Any external Python packages, Django modules, or libraries (for SAML integration, Azure SDK, LDAP connector, etc.) must be compatible with the Ubuntu server environment.

7. P&G will have to provide non-personal IDs across different user groups, with at least two users in each role, which will include QA approver, SPOC for site access only, SPOC at region level and admin.
8. A separate BLOB storage with PDF files will be provided by P&G for test purposes to not hamper the working versions for approvals.

### 3.4.3 Risk Considerations

- Delayed access to required systems (PingFederate, Azure, LDAP) could impact development timelines.
- Delays in reviews will affect the project completion timeline accordingly.
- Changes in report structure or naming conventions from the source system may require additional rework.

## 3.5 Architecture Diagram




**Solid arrow represent Request**
**Light arrows represent response**

- **External Users** :(QA approver, Manufacturing SPOCs, Admins)
- **Nginx:** Reverse proxy & SSL
- **GUNICORN:** WSGI Server
- **Django Application (Anthology Dashboards):** Reports filtering(pass/fail), Inline PDF Viewer, Downloads Option, Daily Report Count, Document Lock Status, QA Commenting Interface, Notification Dispatcher
- **Notification Flows:**
  - **Fail:** Site and Regional SPOCs
  - **Pass:** Site SPOC only

# 4. Business Requirements

## 4.1 Overview

The following Business Requirements (BRs) outline the functional and non-functional expectations for the Anthology Dashboard Application, ensuring that the solution aligns with the project's business objectives of centralizing QA report management, enabling role-based access, and ensuring compliance through standardized workflows.

## 4.1.1 Functional Requirements

| **Functional Requirements** | | |
|---|---|---|
| **BR-01: Secure User Authentication** | | |
| **Objective:** Ensure only authorized corporate users can access the dashboard through PingFederate SSO. | | |
| **ID** | **Requirement Title** | **Detailed Description** |
| BR-01.1 | Federated SSO Integration | 1. The application will integrate with PingFederate SAML 2.0 for Single Sign-On (SSO) authentication.<br>2. Upon accessing the application, users will be redirected to the corporate login page for credential validation.<br>3. User credentials will be authenticated through the organization's Identity Provider (IdP) managed by PingFederate.<br>4. Once authentication is successful, the user will be redirected back to the application with a valid SAML assertion.<br>5. The application will extract user roles and permissions from the SAML attributes to determine access levels.<br>6. Unauthorized users or failed authentication attempts will be redirected to an error or access denied page. |
| BR-01.2 | SAML Assertion Validation | 1. The system will validate the SAML assertion signature using the Identity Provider's (IdP) public certificate.<br>2. The system will reject any SAML responses that are invalid, tampered with, or expired. |

| | | |
|---|---|---|
| | | 3. The system will verify the integrity and authenticity of the assertion before granting access.<br>4. The system will check the validity period (`NotBefore` and `NotOnOrAfter` conditions) to prevent replay attacks.<br>5. Any signature mismatch or untrusted certificate will result in authentication failure and appropriate error handling will be implemented. |
| BR-01.3 | Dynamic Attribute Retrieval | 1. Upon successful authentication, the system will extract user attributes from the SAML claims.<br>2. The extracted attributes will include:<br>a. Username<br>b. Email address<br>c. User roles<br>d. Assigned sites<br>e. Assigned regions<br>3. The system will store these attributes in session variables for use throughout the user's session.<br>4. The session data will be used to control access, personalize views, and enforce authorization rules within the application.<br>5. The system will ensure secure handling of session variables, preventing unauthorized access or modification.<br>6. All session variables will expire automatically upon user logout or session timeout. |
| BR-01.4 | Session Security | 1. All user sessions will be encrypted to protect sensitive information during transmission and storage.<br>2. Sessions will be time-bound with a 30-minute idle timeout to enhance security. |

| | | |
|---|---|---|
| | | 3. Upon reaching the idle timeout limit, the system will automatically terminate the session and redirect the user to the login page. |
| BR-01.5 | Federated Logout | 1. The system will initiate a PingFederate global logout process upon user logout.<br>2. The logout request will be sent to the SSO_LOGOUT_URI endpoint configured in PingFederate.<br>3. This action will terminate all active federated sessions associated with the authenticated user. |
| BR-01.6 | Authentication Logging | 1. The system will capture every login and logout attempt in the Access Log.<br>2. Each log entry will include the following details:<br>a. Timestamp of the event<br>b. User ID associated with the attempt<br>c. Action of the user<br>d. Subject of the Action<br>3. The system will record both successful and failed authentication attempts.<br>4. Access logs will be securely stored and protected from unauthorized modification or deletion.<br>5. The logs will be retained as per organizational audit and compliance policies. |
| **BR-02: Role-Based Access Control (RBAC)** | | |
| **Objective:** Enforce fine-grained authorization aligned with user roles and assigned scopes. | | |
| **ID** | **Requirement Title** | **Detailed Description** |
| BR-02.1 | Role Definition | 1. The system will define and manage user roles for access control and authorization.<br>2. The following roles will be configured within the system:<br>a. Admin – |

| | | |
|---|---|---|
| | | i. Read only access for the entire application.<br>ii. Access to both Access log and Processing log<br>iii. Admin cannot approve (pass / fail) on behalf of QA Approver<br>b. QA Approver<br>i. Ability to approve (Pass / Fail) reports<br>ii. Access to Processing log (no access to 'Access log')<br>c. Manufacturing SPOC (Viewer)<br>i. Read only access to application with ability to view only records pertaining to respective site / region<br>ii. No access to logs<br>3. Each role will have distinct access privileges aligned with business and compliance requirements.<br>4. The system will enforce role-based access control (RBAC) across all modules and functionalities. |
| BR-02.2 | Permission Enforcement | 1. Role permissions will be enforced on server-side. Unauthorized access to restricted pages will be prevented. |
| BR-02.3 | QA Approver Privileges | 1. QA Approvers can view and approve all draft reports and view all finalized reports. |
| BR-02.4 | Manufacturing SPOC Privileges | 1. Manufacturing SPOCs can view only finalized reports belonging to their assigned sites or regions. |
| BR-02.5 | Admin Privileges | 1. Admins can view all logs, metadata, and all reports but cannot modify report content or approvals. |
| BR-02.6 | Dynamic Scope Filtering | 1. Site and region filters must be dynamically applied based on SAML attributes at login. |
| **BR-03: Report Retrieval and Display** | | |
| **Objective:** Securely fetch, list, and view reports stored in Azure Blob Storage. | | |

| ID | Requirement Title | Detailed Description |
|---|---|---|
| BR-03.1 | Secure Azure Connection | 1. The system will connect to Azure Blob Storage using a secure SAS token or service principle over HTTPS. (More likely to be done using personal access tokens - pending final decision from platform owner). |
| BR-03.2 | Report Listing | 1. Reports will be displayed in reverse chronological order with filters for site, date range, and status (Draft/Final). |
| BR-03.3 | Report Viewing | 1. Users will view reports in embedded PDF viewer. |
| BR-03.4 | Report Download | 1. Authorized users can download reports as PDFs. |
| **BR-04: Report Approval Workflow** | | |
| **Objective:** Manage controlled QA review and approval of draft reports. | | |
| **ID** | **Requirement Title** | **Detailed Description** |
| BR-04.1 | Draft Report Access | 1. Only QA Approvers can set Pass / Fail values for a report<br>2. Administrators can view only |
| BR-04.2 | Approval Action | 1. Approvers will mark each draft as **Pass** or **Fail** (not Approve/Reject). |
| BR-04.3 | Approval Metadata | 1. Approval decisions will store approver ID, time stamp, and pass/fail status in the database. |
| BR-04.4 | Locking Mechanism | 1. Once a report is marked **Pass/Fail**, it becomes read-only and visible to manufacturing SPOCs as Final. |
| BR-04.5 | Notification Trigger | 1. Upon approval, the system will generate a notification or mail event to recipients defined in the metadata/PDF file. |
| **BR-05: Recipient Management (LDAP Integration)** | | |
| **Objective:** Enable accurate mail distribution list management through LDAP. | | |
| **ID** | **Requirement Title** | **Detailed Description** |
| BR-05.1 | LDAP Directory Integration | 1. The system will connect securely to the corporate LDAP directory for reading group memberships. |

| | | |
|---|---|---|
| BR-05.2 | Recipient Search | 1. QA Approvers are not required to search LDAP groups manually.<br>2. When a QA Approver marks a report as 'Pass':<br>a. The system will automatically pre-populate the distribution list using the LDAP group membership for the site.<br>3. When a QA Approver marks a report as 'Fail':<br>4. The system will automatically pre-populate the distribution list using the LDAP group membership for both the site and the region. |
| BR-05.3 | Add/Remove Recipients | 1. There will be default set of users from LDAP in a particular category that will be part of the recipient list<br>2. QA Approver working in the specific record will have the ability to add more recipients to the mailing list manually.<br>3. QA Approver will have the ability to remove recipients he has added manually, however will not have the ability to remove the default recipients<br>4. QA Approver will not have the ability to make any changes to the LDAP default recipients list. |
| BR-05.5 | Distribution Validation | 1. Before sending, the system will validate all email addresses (syntax + existence). |
| **BR-06: Audit and Activity Logging** | | |
| **Objective:** Provide full traceability of all user actions. | | |
| **ID** | **Requirement Title** | **Detailed Description** |
| BR-06.1 | Centralized Logging | 1. Every action performed in the application will be logged in as an audit log. This includes: User login, view, download, approve, mail change – will be recorded in a centralized Access Log. |
| BR-06.2 | Log Fields | 1. Each log entry will include the following fields: User ID, role, action type, timestamp, report ID, and originating IP. |

| | | |
|---|---|---|
| BR-06.3 | Type of Logs | 1. There are two types of logs, namely:<br>a. Processing Log<br>b. Access Log |
| BR-06.4 | Log Accessibility | 1. Logs can be accessed by Admin and QA Approvers only. SPOC user will have no access to logs<br>2. Admin user can access both processing logs and access logs<br>3. QA Approver can access only processing log and will not have access to access logs |
| **BR-07: Dashboard and User Interface** | | |
| **Objective:** Deliver a responsive, intuitive dashboard. | | |
| **ID** | **Requirement Title** | **Detailed Description** |
| BR-07.1 | Responsive Design | 1. Built using **Bootstrap 5** for seamless viewing on desktop. |
| BR-07.2 | Metrics Display | 1. **No KPIs required** for the dashboard interface.<br>2. **QA Approver will receive visual notifications** for outstanding draft reports.<br>3. **Notification appears via the "Approvals" navigation element**.<br>4. **CSS-based indicators** will be used to highlight pending reviews.<br>**Examples include:**<br>a. A distinct background style<br>b. A badge or red dot next to the "Approvals" icon<br>5. These visual cues ensure that the QA Approver can quickly identify reports needing attention. |
| BR-07.3 | Filtering and Search | 1. Users can filter by Region, Site, Batch, Product, Pass, Start Date, End Date, Approved By, Approved On. |
| BR-07.4 | Navigation | 1. Provide intuitive top-menu navigation with quick access to Reports, Approvals, Logs. |
| BR-07.5 | Accessibility | 1. UI will comply with P&G brand guidelines. |
| **BR-08: Data Storage and Management** | | |
| **ID** | **Requirement Title** | **Detailed Description** |

| | | |
|---|---|---|
| BR-08.1 | Database Design | 1. All metadata, approvals, logs, and process data shall reside in an on-premises MySQL database. |
| BR-08.2 | ORM Integrity | 1. Use Django ORM migrations to enforce schema consistency and referential integrity. |
| BR-08.3 | Backup | 1. Daily backups will be configured and managed by P&G |
| BR-08.4 | Data Validation | 1. All insert/update actions will include server-side validation for required fields and data types. |
| **BR-09: Security and Compliance** | | |
| **ID** | **Requirement Title** | **Detailed Description** |
| BR-09.1 | Encryption in Transit | 1. All connections (SSO, LDAP, Azure, MySQL) will use HTTPS, (LDAP done by LDAPS) |
| BR-09.2 | Credential Protection | 1. Secrets (DB, LDAP user) must be stored in encrypted vaults, never in plain text. |
| BR-09.3 | Access Control | 1. Apply the least-privilege principle to all service accounts. |
| BR-09.4 | Compliance Audit | 1. System will comply with P&G data handling and privacy standards. |
| **BR-10: Reporting and Monitoring (Optional)** | | |
| **ID** | **Requirement Title** | **Detailed Description** |
| BR-10.1 | Summary Reports | 1. Admins can generate reports summarizing approvals, site statistics, and user activity as required. |
| BR-10.2 | Export Options | 1. Export feature that supports configurable date ranges and filters. |
| BR-10.3 | Error Logging | 1. All failed data fetch or system errors will be captured in error logs. |

### 4.1.2 Non-Functional Requirements

| | | |
|---|---|---|
| **BR-11: Performance** | | |
| **ID** | **Requirement Title** | **Detailed Description** |
| BR-11.1 | Query Optimization | 1. Use indexed columns and pagination to maintain response times. |
| **BR-12: Usability** | | |

| ID | Requirement Title | Detailed Description |
|---|---|---|
| BR-12.1 | Branding Compliance | 1. Follow P&G branding colors, fonts, logos. |
| BR-12.2 | Cross-Device Optimization | 1. Support the latest version of Chrome browser. |
| **BR-13: Maintainability** | | |
| **ID** | **Requirement Title** | **Detailed Description** |
| BR-13.1 | Code Modularity | 1. Django best practices will be used. |
| BR-13.2 | Version Control | 1. Use Git branching model with peer review before merging. |
| **BR-14: Deployment** | | |
| **ID** | **Requirement Title** | **Detailed Description** |
| BR-15.1 | Deployment Stack | 1. Deploy on a secure on-prem Ubuntu server. |

## 4.2 Design Considerations

a. Development Environment

- **Framework**: Django (Python 3.10+)
- **Database**: MySQL (on-premises)
- **Authentication**: PingFederate SSO using SAML 2.0
- **Directory Services**: LDAP for user roles and recipient lists
- **PDF Storage**: Azure Blob Storage
- **Frontend**: Bootstrap 5 (responsive UI)
- **Version Control**: GitHub (P&G internal)

b. Deployment Environment

- **Host OS**: Ubuntu Server (P&G network)
- **Web Server**: Nginx (reverse proxy, SSL termination)
- **Application Server**: Gunicorn (WSGI server)
- **HTTPS/SSL**: Enforced with internal certificate
- **Reverse Proxy Flow**:
  Client -> Nginx -> Gunicorn -> Django App

c. User Environment

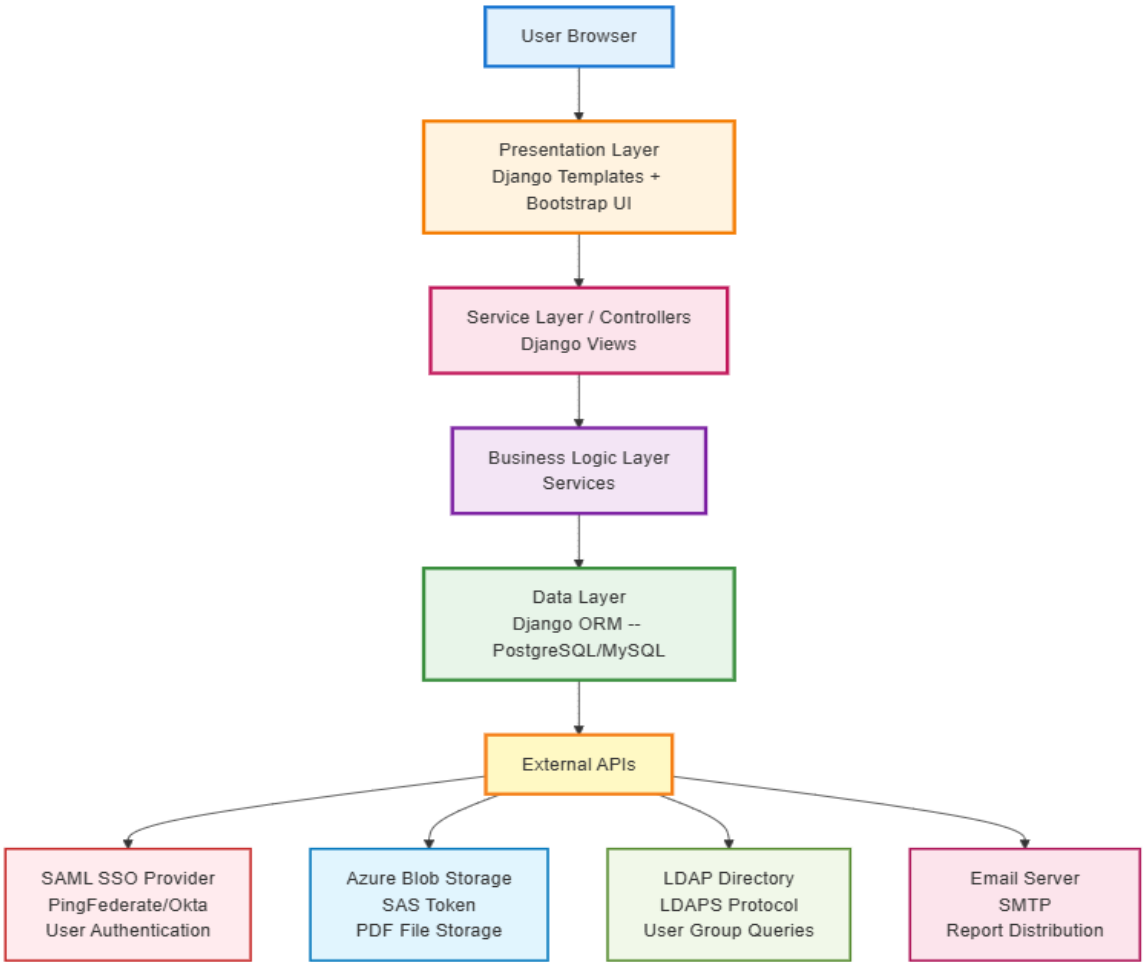
- **Browsers**: Chrome, Firefox (latest versions)
- **Devices**: Desktop
- **VPN Access**: Required for all roles (internal access only)

# 5. Low Level Design (LLD)

The Low-Level Design focuses on component-level architecture, class relationships, APIs, and UI wireframes.

Django ORM will be used to define the DB models and to migrate into MySQL.

## 5.1 Application Architecture (Component View)




**1. User Browser** – Client interface accessed via modern browsers (Chrome, Edge, Firefox).

**2. Presentation Layer** – Django Templates + Bootstrap UI providing responsive web pages and consistent layout styling.

**3. Service Layer / Controllers** – Django Views handle incoming HTTP requests, route to appropriate business logic, and return rendered responses.

**4. Business Logic Layer** – Core backend services implementing main functionalities:

- Authentication & Authorization
- Reports Management
- Approvals Workflow

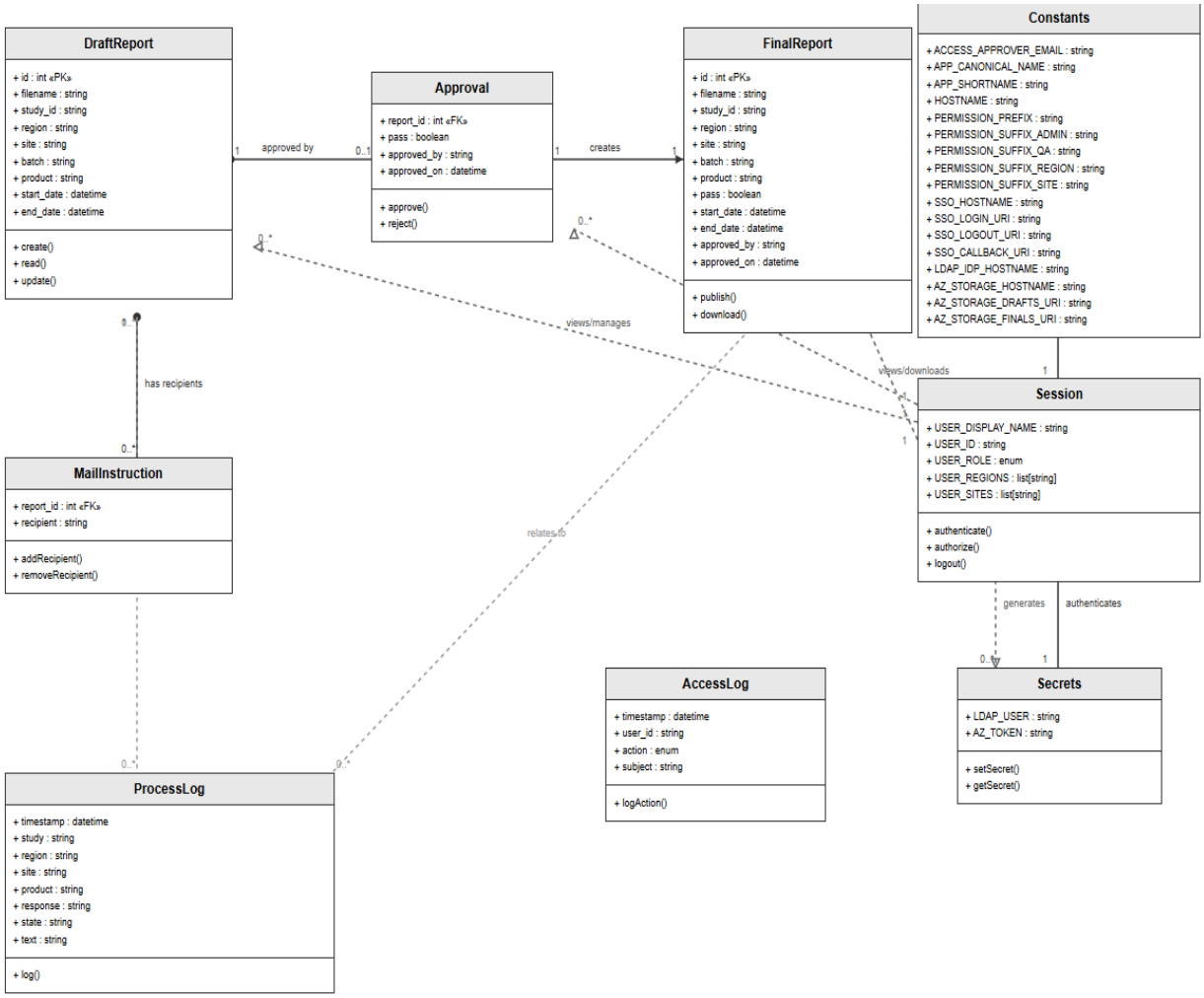
- Logging & Audit
- Filtering & Sorting
- Mail Handling
- Document viewers

**5. Data Layer** – Django ORM integrated with **MySQL** for relational data.

**External Systems (4 Integrations):**

1. **SAML SSO (PingFederate )** – Handles enterprise user authentication and role mapping via SAML 2.0.
2. **Azure Blob Storage** – Secure PDF file storage using SAS tokens for controlled access.
3. **LDAP Directory** – Performs user group lookups for permission and email list generation.
4. **Email Server (SMTP)** – Sends report notifications and distribution of emails.

## 5.2 Class Diagram:




## 5.2.1 DraftReport

**Purpose:** Stores metadata for unapproved study reports.
**Attributes:**

- `id` (PK), `filename` (Azure draft PDF), `study_id`, `region`, `site`, `batch`, `product`, `start_date`, `end_date`.
  **Methods:** `create()`, `read()`, `update()` – CRUD for drafts.
  **Relationships:**
- 1–1 with **Approval** (each draft approved once).
- 1–* with **MailInstruction** (multiple recipients).
- 1–1 with **FinalReport** (draft becomes final).

### 5.2.2 Approval

**Purpose:** Records QA Approver assessments for draft reports and triggers downstream final report generation.

**Attributes:**
report_id (FK to DraftReport), pass, approved_by, approved_on

**Methods:**
approve()

**Behavior Notes:**

a. Submitting an approval decision does **not** immediately create a FinalReport entry.
b. The decision and its associated distribution list are **stored first**.
c. A **downstream data integration process on Azure**:
    i. Generates the FinalReport PDF.
    ii. Inserts the corresponding FinalReport record.
d. **Email notifications** are triggered **only after** the FinalReport record exists, via a **scheduled task**.
e. The Final Report object **should not be created immediately** when the QA Approver marks a report as Pass/Fail (Pass -The site, Fail-Both the site and the region of LDAP group members).
f. This is because the **final PDF report is not yet generated** on Azure at that stage.

**Relationships:**
Belongs to DraftReport

### 5.2.3 Final Report

**Purpose:** Represents the completed report generated after QA approval, available for viewing and download once processed on Azure.

**Attributes:**
Same as DraftReport + pass, approved_by, approved_on

**Methods:**
publish() (mark as visible when available)
download() (retrieve PDF)

**Behavior Notes:**
FinalReport records are not created directly by the application. They are inserted by the downstream Azure process once the final PDF has been generated using both the approval decision and final data pushed from the dashboard.

**Relationships:**
Linked to DraftReport
Accessible to authorized users for viewing and download

### 5.2.4 Mail Instruction

**Purpose:** Stores the full distribution list for a published report that has been confirmed by the QA Approver. This includes both recipients sourced from LDAP at approval time and any additional recipients manually added by the QA Approver.

**Attributes:**

- report_id (FK to DraftReport)
- recipient (user ID or email)
- source_type (LDAP or Custom) [optional but recommended for traceability]
- timestamp_added

**Methods:**

- addRecipient()
- removeRecipient()

**Behavior Notes:**
The mail distribution task will not refresh or re-query LDAP before sending. It will rely entirely on the entries recorded in this table to ensure the final logged distribution list matches what the QA Approver reviewed.

**Relationships:**

- Many per DraftReport (0..*)

### 5.2.5 Process Log

**Purpose:** Captures data processing events from Databricks.
**Attributes:**

- `timestamp`, `study`, `region`, `site`, `product`, `response`, `state`, `text`.
  **Methods:** `log()` – record event.
  **Relationships:**
- Relates to **DraftReport** via shared study/site IDs.

### 5.2.6 Access Log

**Purpose:** Logs user actions (login, view, download, approve, mail).
**Attributes:**

- `timestamp`, `user_id`, `action` (enum), `subject`.
  **Methods:** `logAction()`.
  **Relationships:**
- Linked to **Session** via `user_id`.

### 5.2.7 Session

**Purpose:** Represents user authentication and access scope.
**Attributes:**

- `USER_DISPLAY_NAME`, `USER_ID`, `USER_ROLE` (Unauthenticated / Admin / Approver / Viewer), `USER_REGIONS`, `USER_SITES`.
  **Methods:** `authenticate()`, `authorize()`, `logout()`.
  **Relationships:**
- Authenticates via **Secrets**.
- Generates **AccessLog** entries.
- Views/downloads **FinalReport**.

### 5.2.8 Secrets

**Purpose:** Stores encrypted credentials for external integrations.
**Attributes:**

- `LDAP_USER` (identity provider), `AZ_TOKEN` (Azure storage).
  **Methods:** `setSecret()`, `getSecret()`.
  **Relationships:**
- Authenticates **Session**.

### 5.2.9 Constants

**Purpose:** Stores environment and configuration constants.
**Examples:**

- App info: APP_CANONICAL_NAME, HOSTNAME.
- Roles: PERMISSION_SUFFIX_ADMIN, PERMISSION_SUFFIX_QA, etc.
- SSO: SSO_LOGIN_URI, SSO_CALLBACK_URI.
- Azure: AZ_STORAGE_DRAFTS_URI, AZ_STORAGE_FINALS_URI.

**Relationships:**

- Referenced by **Session**, **Secrets**, **FinalReport** for logic/config.

## 5.3 Wireframe:

Here is the point of view on the reference of wireframe document:

### 5.3.1 Login & Authentication:

- **Flow:** User → SSO → SAML Auth → Parse MemberOf → Assign Role → Create Session → Redirect to Dashboard
- **Roles via SAML MemberOf:**
  - [PERMISSION_PREFIX]-[PERMISSION_SUFFIX_ADMIN] → **Admin**
  - [PREFIX]-QA-APPROVER → **QA Approver**
  - [PERMISSION_PREFIX]-[PERMISSION_SUFFIX_REGION]-[NAME] / [PERMISSION_PREFIX]-[PERMISSION_SUFFIX_SITE]-[NAME] → **Viewer (Manufacturing SPOC)** with access scoped to the specified region or site

  Where the placeholders (PERMISSION_PREFIX, PERMISSION_SUFFIX_ADMIN, PERMISSION_SUFFIX_QA, PERMISSION_SUFFIX_REGION, PERMISSION_SUFFIX_SITE) are configurable string values stored in the **Constants** table so changes to corporate naming conventions can be supported without code updates.

- **Session Stores:** USER_DISPLAY_NAME, USER_ID, USER_ROLE, USER_REGIONS, USER_SITES
- **Logout:** Clears session data and cache, then redirects to PingFederate SSO logout URL (SSO_LOGOUT_URL).

Authentication events (login/logout/failure) are recorded in the AccessLog model for audit traceability.

### 5.3.2 Common UI Elements:

- **Banner:** App logo and title, Logged-in user display name, and logout button (redirects via SSO logout)

- **Navigation Menu:**

| Menu Item | Visible To | URL | Purpose |
|---|---|---|---|
| Reports | All Roles | /reports | View and download approved final reports |
| Approvals | Admin & QA Approver | /drafts | Review and approve draft reports |
| Logs | Admin & QA Approver | /logs/processing | View processing and access logs |

### 5.3.3 Reports Page:

- **Purpose:** View/download approved final reports.
- **Features:**
- Filtering, sorting, infinite scroll.
- Infinite scroll or pagination (depending on data size)
- "No records found" placeholder for empty results
- Access:
    - **Admin/QA Approver:** Can view all final reports
    - **Manufacturing SPOC:** Restricted by assigned region/site
- **Individual Report View:**
    - PDF viewer from Azure Storage
    - Metadata panel (Region, Site, Batch, Product, Start Date, End Date, Approved By, Approved On)
    - Download option
    - "Document Unavailable" if file fetch fails

### 5.3.4 Approvals Page:

- **Purpose:** QA review and approval of draft reports
- **Access:** Admin and QA Approver ONLY (Manufacturing SPOC redirected to /forbidden)
- **Draft Table:** Same filtering/sorting as Reports.
- **Review Interface:**
    - View draft PDF
    - QA Approver selects **Pass/Fail**,
    - Edits manual mail list,
    - Confirm approval decision.

- **Mail List Editor Modal:**
  - Auto-fetched via LDAP (site/region-based)
  - Option to add/Remove manual recipients
- **Workflow:** Review → Select Result → Confirm → Wait for Final Report → Send Mail notifications→ Log Action

| Selection | Recipients Included |
|---|---|
| **Pass** | Site members only |
| **Fail** | Site members + Region members |

### 5.3.5 Logs Pages:

- **Processing Logs:** (Admin & QA Approver)
  - **Purpose:** To monitor system events and user actions.
  - Show timestamps, process names, status, and error text (if any)
- **Access Logs:** (Admin only)
  - **Purpose**: Record user-level actions for auditing
  - Full filtering, sorting, and infinite scroll for auditing and security tracking.
  - **Action Types & Subject Format:**

| Action | Subject Format | Example |
|---|---|---|
| Login | [IP Address] | 192.168.1.100 |
| View | [type]:[id] | report:12345 |
| Download | [type]:[id] | draft:67890 |
| Approved | [id]:[result] | 12345:pass |
| Mail | [id]:[recipient] | 12345:user@company.com |

## 5.4 Technical Specifications:

- **Endpoints:** `/reports`, `/drafts`, `/logs/processing`, `/logs/access` with pagination, filtering, sorting.
- **Authorization Rules:**
  - Reports → All roles
  - Drafts & Processing Logs → Admin/QA Approver
  - Access Logs → Admin only

- **Filtering Logic:** Enforces scope for Manufacturing SPOC at query level.

### 5.4.1 Remote Resources:

- **PDF Storage:** Azure Blob (SAS token secured).
- **Mail List Source:** LDAP queries for site/region members.
- Authentication: PingFederate (SAML 2.0)

### 5.4.2 Security Features:

- **SSO (SAML 2.0):** Centralized authentication via PingFederate.
- **RBAC:** Three roles (Admin, QA Approver, SPOC) with endpoint-level enforcement
- **Scope-based access:** Prevents unauthorized data viewing.
- **Session Management:** Secure Django sessions.
- **Audit Logging:** Immutable logs, admin-only visibility.
- **Encrypted Secrets & SAS Tokens:** Protects credentials and resources.

## 5.5 User Journey:

### 5.5.1 Manufacturing SPOC Journey:

1. *Login → SSO Authentication*
2. *Session created with USER_SITES = ["Site A", "Site B"]*
3. *Navigate to Reports*
4. *See only reports where* site belongs to USER_SITES
5. *Click Report → View PDF /Download PDF*
6. *Logout*

### 5.5.2 QA Approver Journey:

1. *Login → SSO Authentication* (`SESSION.USER_ROLE = QA Approver`)
2. *Navigate to Approvals page*
3. *View all draft reports*
4. *Click on a draft → Open Review Draft page -> PDF viewer loads*
5. Click **Download PDF**
6. Click Approve → Opens Approval Dialog
7. In dialog:
   i. **Edit Mail List** → Add/Remove recipients (custom emails allowed)
   ii. **Click Confirm** to finalize recipients

8. After confirmation → Select **Pass** or **Fail** outcome
9. **Approval submitted** → Draft becomes **wait for Final Reports**
10. **Emails sent** to all recipients
11. **Action logged** in Access Log
12. **Logout**

### 5.5.3 Admin Journey:

*1. Login → SSO Authentication (role = Admin)*
*2. Navigate to Reports → View all reports*
*3. Navigate to Approvals → View all drafts (read only and cannot approve )*
*4. Navigate to Logs → Processing → Check system events*
*5. Switch to Access Logs tab*
*6. Filter by user to audit specific activity*
*7. Export findings for compliance report*
*8. Logout*

## 6. Conclusion

The Anthology Dashboard Application represents a significant step forward in modernizing and streamlining the Monitoring Shave Test (MST) reporting process within the Gillette "Blades & Razors" capability. By automating the retrieval, approval, and distribution of QA reports, the solution eliminates manual inefficiencies, reduces process delays, and ensures data integrity across all manufacturing sites.

The application's Django-based architecture, integrated with PingFederate SSO, Azure Blob Storage, LDAP, and MySQL, provides a secure and scalable enterprise-grade platform. The introduction of role-based access, centralized storage, and automated logging ensures full compliance with organizational governance, audit, and security requirements.

Beyond operational improvements, the Anthology Dashboard delivers strategic business value by:

- Establishing a single source of truth for QA report data.
- Enhancing collaboration between QA teams and manufacturing units.
- Reducing manual workload, improving transparency, and increasing approval efficiency.
- Ensuring traceability and accountability through activity logs and version-controlled data.

With this implementation, the organization will gain a centralized, reliable, and future-ready QA reporting ecosystem, laying out the foundation for further automation, analytics, and AI-driven quality insights in subsequent phases.